

Dungeons and JavaScript

A Tornada do Guerreiro
Front-End



GABRIEL DESTERRO

Guia Rápido de JavaScript:

Sintaxe Essencial com Exemplos Práticos

JAVASCRIPT É UMA LINGUAGEM DE PROGRAMAÇÃO ESSENCIAL PARA O DESENVOLVIMENTO WEB, PERMITINDO CRIAR INTERATIVIDADE E FUNCIONALIDADES DINÂMICAS EM SITES. USADO TANTO NO LADO DO CLIENTE (NAVEGADOR) QUANTO NO SERVIDOR (VIA NODE.JS), JAVASCRIPT OFERECE FLEXIBILIDADE PARA MANIPULAR ELEMENTOS DA PÁGINA, FAZER REQUISIÇÕES A SERVIDORES, VALIDAR FORMULÁRIOS E MUITO MAIS. SUA SINTAXE SIMPLES E PODEROSA, ALIADA À VASTA COMUNIDADE DE DESENVOLVEDORES, TORNA-O UMA DAS LINGUAGENS MAIS POPULARES E INDISPENSÁVEIS PARA CRIAR EXPERIÊNCIAS RICAS E RESPONSIVAS NA WEB.

SUMÁRIO



- DECLARANDO VARIÁVEIS.....4
- FUNÇÕES.....6
- CONDICIONAIS.....8
- LOOPS.....10
- OBJETOS E ARRAYS.....12
- MANIPULAÇÃO DE DOM.....14
- PROMISES E ASYNC/AWAIT.....16
- AGRADECIMENTOS.....18

01



DECLARANDO VARIÁVEIS

Declarando Variáveis

VARIÁVEIS ARMAZENAM DADOS QUE PODEM SER USADOS NO CÓDIGO. EXISTEM TRÊS PALAVRAS-CHAVE PRINCIPAIS PARA DECLARAR VARIÁVEIS:

VAR: ESCOPO GLOBAL OU DE FUNÇÃO (MENOS USADO ATUALMENTE).

LET: ESCOPO DE BLOCO, MAIS MODERNA E RECOMENDADA.

CONST: ESCOPO DE BLOCO PARA VALORES CONSTANTES (NÃO PODEM SER REATRIBUÍDOS).

Declarando Variáveis

```
1 let nome = "João"; // Pode ser reatribuído
2 const idade = 30; // Valor fixo
3 var cidade = "Rio"; // Uso legado
4
5 console.log(nome, idade, cidade);
```

02

FUNÇÕES

Funções

FUNÇÕES EXECUTAM BLOCOS DE CÓDIGO. PODEM SER DECLARADAS DE DIVERSAS FORMAS.

DECLARAÇÃO TRADICIONAL

```
1 function saudacao(nome) {  
2     return `Olá, ${nome}!`;  
3 }  
4 console.log(saudacao("Maria"));
```

FUNÇÃO ANÔNIMA

```
1 const soma = function(a, b) {  
2     return a + b;  
3 };  
4 console.log(soma(5, 3));
```

ARROW FUNCTION

```
1  
2 const multiplicar = (a, b) => a * b;  
3 console.log(multiplicar(4, 7));  
4
```

03

CONDICIONAIS

Condiçõais

CONDICIONAIS SÃO USADOS PARA EXECUTAR DIFERENTES BLOCOS DE CÓDIGO COM BASE EM CONDIÇÕES ESPECÍFICAS. ELES PERMITEM QUE O PROGRAMA TOME DECISÕES E EXECUTE AÇÕES DEPENDENDO DO RESULTADO DAS CONDIÇÕES AVALIADAS.

```
1  const hora = 15;
2
3  if (hora < 12) {
4      console.log("Bom dia!");
5  } else if (hora < 18) {
6      console.log("Boa tarde!");
7  } else {
8      console.log("Boa noite!");
9  }
```



LOOPS

Loops

LOOPS SÃO ESTRUTURAS QUE PERMITEM REPETIR AÇÕES DE FORMA CONTROLADA ATÉ QUE UMA CONDIÇÃO ESPECÍFICA SEJA ATENDIDA.

LOOP COM FOR

```
1 for (let i = 0; i < 5; i++) {  
2     console.log(`Contagem: ${i}`);  
3 }
```

LOOP COM WHILE

```
1 let contador = 0;  
2 while (contador < 5) {  
3     console.log(`Contagem: ${contador}`);  
4     contador++;  
5 }
```

LOOP COM FOR...OF

```
1 const frutas = ["Maçã", "Banana", "Laranja"];  
2 for (const fruta of frutas) {  
3     console.log(fruta);  
4 }
```

05



OBJETOS E ARRAYS

Objetos e Arrays

OBJETOS SÃO ESTRUTURAS QUE ARMAZENAM DADOS EM PARES DE CHAVE-VALOR, PERMITINDO ACESSO FÁCIL ÀS INFORMAÇÕES POR MEIO DE SUAS CHAVES. ARRAYS, POR OUTRO LADO, SÃO LISTAS ORDENADAS DE ELEMENTOS, ACESSADOS POR ÍNDICES NUMÉRICOS. OBJETOS E ARRAYS ARMAZENAM COLEÇÕES DE DADOS.

OBJETOS

```
1 const frutas = ["Maçã", "Banana", "Laranja"];
2 for (const fruta of frutas) {
3     console.log(fruta);
4 }
```

ARRAYS

```
1 const cores = ["Vermelho", "Azul", "Verde"];
2 console.log(cores[1]); // Acessando elemento
```

06



MANIPULAÇÃO DE DOM

Manipulação de DOM

O DOM (DOCUMENT OBJECT MODEL) É UMA REPRESENTAÇÃO ESTRUTURADA DE UM DOCUMENTO HTML, PERMITINDO QUE O JAVASCRIPT ACESSE E MANIPULE OS ELEMENTOS DA PÁGINA. COM ELE, É POSSÍVEL ALTERAR O CONTEÚDO, ESTILOS E ATÉ ADICIONAR OU REMOVER ELEMENTOS DINAMICAMENTE.

EXEMPLO:

```
1 const titulo = document.querySelector("h1");  
2 titulo.textContent = "Novo Título"; // Alterando o texto
```

07



PROMISES E ASYNC/AWAIT

Promises e Async/Await

EM JAVASCRIPT, MUITAS OPERAÇÕES COMO CHAMADAS A APIS, LEITURA DE ARQUIVOS OU TEMPORIZADORES SÃO ASSÍNCRONAS. ISSO SIGNIFICA QUE O CÓDIGO PODE CONTINUAR SENDO EXECUTADO ENQUANTO ESSAS OPERAÇÕES AGUARDAM UMA RESPOSTA. PARA LIDAR COM ESSAS TAREFAS, UTILIZAMOS PROMISES E A SINTAXE ASYNC/AWAIT, QUE TORNAM O GERENCIAMENTO DESSAS OPERAÇÕES MAIS SIMPLES E LEGÍVEL.

PROMISE

```
1 const promessa = new Promise((resolve, reject) => {
2     const sucesso = true;
3     if (sucesso) {
4         resolve("Deu certo!");
5     } else {
6         reject("Deu errado!");
7     }
8 });
9
10 promessa.then(console.log).catch(console.error);
```

ASYNC/AWAIT

```
1 async function obterDados() {
2     try {
3         const resposta = await fetch("https://api.com/dados");
4         const dados = await resposta.json();
5         console.log(dados);
6     } catch (erro) {
7         console.error("Erro ao buscar dados", erro);
8     }
9 }
10 obterDados();
```

AGRADECIMENTOS



Obrigado por ler até aqui!

A TI QUE ENFRENTASTE AS PÁGINAS DESTE GUIA, MEU MAIS SINCERO AGRADECIMENTO.

COMO UM GUERREIRO QUE EMPUNHA SUA ESPADA COM SABEDORIA, AGORA TENS EM TUAS MÃOS O PODER DE DOMINAR OS SEGREDOS DA PROGRAMAÇÃO, DESBRAVANDO AS VASTAS TERRAS DA WEB COM CORAGEM E DESTREZA.

QUE OS CONHECIMENTOS AQUI ADQUIRIDOS SEJAM TUA ARMADURA E QUE A LINGUAGEM DE JAVASCRIPT SEJA TUA FIEL ALIADA NAS BATALHAS DO CÓDIGO. CONTINUE A TRILHAR TEU CAMINHO COM BRAVURA, POIS O VERDADEIRO GUERREIRO NUNCA PARA DE APRENDER E EVOLUIR.

QUE A LUZ DO CONHECIMENTO SEMPRE TE GUIE.

COM HONRA E GRATIDÃO, GABRIEL DESTERRO.

GitHub

LinkedIn

Instagram